Programming for Data Science (BCSE207L) - Course Project: Video Reception Analyzer (VRA)

Project Overview: VRA is a full-stack, multi-language sentiment analysis system that takes any YouTube video URL and returns a real-time audience sentiment score powered by a custom deep-learning model trained on real review data.

Student Details:

YASH PODDAR (23BDS0195)

AAYUSH SOOD (23BDS0177)

1. Problem Statement

Requirement Setup: Identify a real-world data science problem and implement an end-to-end pipeline in R.

Implementation (VRA): The real-world problem addressed is analyzing true audience reception to YouTube videos beyond simple like-counts. We built the Video Reception Analyzer, an end-to-end pipeline that fetches live YouTube comments, preprocesses them using R, and applies a custom Multi-Signal Sentiment Fusion (MSSF) neural network in Python to classify sentiment (positive, neutral, negative). The data engineering and offline text mining baselines (Naive Bayes) were completely implemented in R, while modern deep learning and frontend frameworks handle the production predictions and dashboarding.

2. Dataset Requirements

Requirement Setup: Public reliable sources, min 10,000 rows, min 8 attributes, numeric + categorical data.

Implementation (VRA): The training dataset used is the UCI Sentiment Labelled Sentences dataset (from Amazon, Yelp, and IMDb). It contains a baseline of 10,000 sentences. We augment this data structurally during the R pipeline to build rich features (word counts, TF-IDF / DTM representation, etc.). For live evaluation, we actively scrape data via the YouTube API v3, dynamically creating test datasets that include text, numeric engagement signals

(like_count, reply_count), and categorical/text properties (author, text). This fulfills the >10,000 row requirement and provides more than 8 attributes across our processed relational tables.

3. API Integration

Requirement Setup: Integrate at least one API for data ingestion, handle auth/pagination, convert JSON to R DF.

Implementation (VRA): The system integrates the YouTube Data API v3. The component 'r/api_fetch.R' accepts a video URL, authenticates using an API key, and makes paginated GET requests to the commentThreads endpoint. It uses 'httr' and 'jsonlite' to unpack standard JSON responses into a tidy R dataframe, successfully managing token pagination and API quotas.

4. R Programming Requirements

Requirement Setup: API extraction, cleaning, EDA, Modelling (with ggplot2 and metrics).

Implementation (VRA):
- Data Preparation: R scripts ('r/04_apply_preprocessing.R') build an R corpus using the 'tm' package, handling lowercase normalization, punctuation/number stripping, and stopword removal before generating a Document-Term Matrix (DTM).
- EDA: Scripts like 'r/evaluate.R' generate robust summary statistics and at least 5 meaningful ggplot2 visualizations (e.g., sentiment donut charts, class confidence histograms, probability heatmaps, and violin plots of engagement).
- Modelling: We built a Naive Bayes model baseline in R using 'e1071::naiveBayes', achieving 86% accuracy and comparing it via 5-fold cross validation against a Logistic Regression model.

5. Power BI (with R Visuals)

Requirement Setup: R scripts inside Power BI for advanced custom visuals.

Implementation (VRA): The backend constantly updates a cumulative export file ('exports/dashboard_data.csv'). This tabular dataset is directly plugged into Power BI. Within Power BI, custom R visual scripts generate advanced analytical plots using 'ggplot2' to display running averages of video sentiment, probability distributions, and the correlation between engagement and model confidence.

6. GitHub Requirement

Requirement Setup: Commits on main/development, Detailed README, no API keys pushed.

Implementation (VRA): The project features a detailed set of markdown files (README.md, EXPLANATION.md, DOCKER.md, etc.) outlining setup and project structure. No hardcoded secrets exist; a '.env' file combined with '.gitignore' keeps the YouTube API Data Key completely local.

7. Docker Containerization

Requirement Setup: Publish containerized application (excluding PowerBI) with proper tags.

Implementation (VRA): The entire application (Next.js frontend, Node Node backend orchestrating R, and Python ML Server) is fully containerized. A 'docker-compose.yml' unifies these 3 services. These images are correctly version-tagged for potential deployment to Docker Hub.